

SEVA SANKALP — END-TO-END TECH STACK & FEATURE SPECIFICATION

Comprehensive Technical Notes on Frontend, Backend, Database, Security & Feature Implementations

B.Tech CSE Project Review Dossier | MVGR College of Engineering (Autonomous) | Python 3.10+ & Flask Architecture

1. CORE PROGRAMMING LANGUAGES & OPERATING RUNTIMES

- **Python (Version 3.10+ / 3.13):** Primary backend programming language. Executes business logic, mathematically computes Haversine distances (`math`), calculates sliding-window cooldowns (`datetime`), enforces statutory regex pattern checks (`re`), and interacts with the ORM layer.
- **JavaScript (ECMAScript 2020 / ES6+):** Primary client-side scripting language. Powers asynchronous Fetch API requests (NGO likes, in-app messaging, dynamic map filtering), client-side coordinate acquisition (`navigator.geolocation`), and DOM mutations without page reloads.
- **HTML5 (HyperText Markup Language 5):** Defines the semantic structure of the presentation tier, including form controls, microdata, canvas/SVG rendering hooks, and accessible UI hierarchies across mobile and desktop viewports.
- **CSS3 (Cascading Style Sheets 3):** Provides responsive layout orchestration using CSS Grid and Flexbox, custom properties (`--bg-primary`, `--accent`), keyframe transitions, and a flash-free dark/light mode engine.
- **SQL (Structured Query Language):** Underlying query language compiled dynamically by SQLAlchemy ORM to manage relational schema integrity, indexed table lookups, and transactional ACID commits.

2. FRONTEND ENGINEERING STACK & FEATURE MAPPING

Frontend Technology	Version / Spec	Platform Features Powered	Technical Role & Implementation Detail
HTML5 & Semantic Web	W3C Recommendation	All Web Pages & Forms	Provides accessible document structure, native form validation (<code>required</code> , <code>pattern</code>), multi-part image upload inputs, and accessibility (WAI-ARIA).
CSS3 & Theme Variables	CSS3 Standards	Global Styling & Theme Toggle	Implements dynamic CSS custom properties for instant dark/light mode switching (<code>[data-theme='dark']</code>) stored in <code>localStorage</code> , eliminating visual flickering.
Bootstrap	v5.3.2 (CDN)	Responsive Grid & Modals	Delivers 12-column responsive layout, interactive modal dialogs (DigiLocker verification, document viewers), alert banners, and mobile navigation toggles.
Leaflet.js	v1.9.4 (Open Source)	Interactive Community Map	Lightweight client GIS engine (42 KB). Binds to HTML div <code>#map</code> , computes viewport tile coordinates $S(x,y,z)$, and plots custom interactive SVG resource pins.
OpenStreetMap (OSM)	Carto Tile Layer	Geospatial Tile Rendering	Serves free raster map imagery via <code>https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png</code> ; eliminates commercial Google Maps API billing and tracking.
FontAwesome	v6.4.0 (SVG/Font)	Iconography & Visual Cues	Renders scalable vector icons including community heart (<code>fa-heart</code>), location pins, verified NGO badges, trust stars, and status indicators.
Vanilla JavaScript (ES6+)	Native Browser Engine	AJAX Likes, Chat & Search	Executes asynchronous <code>fetch()</code> API calls with CSRF token headers, handles dynamic radius slider input events, and performs optimistic UI DOM updates.

3. BACKEND ENGINEERING STACK & FEATURE MAPPING

Backend Technology	Version / Spec	Platform Features Powered	Technical Role & Implementation Detail
Flask (WSGI Framework)	v3.0.x (Python)	Core Web Application Engine	Implements Application Factory Pattern (<code>create_app()</code>) and 8 modular Blueprints (<code>auth</code> , <code>resource</code> , <code>request</code> , <code>search</code> , <code>profile</code> , <code>admin</code> , <code>points</code> , <code>history</code>).
Jinja2 Templating Engine	v3.1.x	Server-Side Rendering (SSR)	Compiles dynamic HTML templates with context dictionaries, provides template inheritance (<code>base.html</code>), and enforces automatic context-aware XSS auto-escaping.

Backend Technology	Version / Spec	Platform Features Powered	Technical Role & Implementation Detail
SQLAlchemy (ORM)	v2.0.x	Data Persistence & Models	Maps Python classes to database tables ('User', 'Resource', 'Request', 'NGOLike'), generates parameterized queries (SQLi immune), and enforces cascading deletes.
Flask-Login	v0.6.x	User Session State Machine	Manages user login state, session cookies ('HttpOnly', 'SameSite=Lax'), user loader callbacks ('@login_manager.user_loader'), and '@login_required' route guards.
Bcrypt	v4.x (Blowfish)	Password Hashing & Auth	Applies Blowfish cipher key expansion with 128-bit random salt ('bcrypt.gensalt()') and cost factor 12 ($2^{12} = 4,096$ rounds) for GPU-proof password security.
Flask-WTF / CSRFProtect	v1.2.x	Form & POST Security	Injects signed HMAC-SHA256 tokens into all POST forms; validates session token matches request token, stopping Cross-Site Request Forgery.
Werkzeug Toolkit	v3.0.x	WSGI Dispatching & Uploads	Handles HTTP request dispatching, file stream buffering, and uses 'secure_filename()' to sanitize uploaded image/document paths against directory traversal.
Python Standard Library	Python 3.10+	Math, Cooldown & Security	'math': Computes Haversine spherical trigonometric distance. 'datetime': Enforces 365d/180d cooldowns & 7-day velocity caps. 're': Validates statutory Darpan ID & PAN regex.

4. DATABASE, PERSISTENCE & STORAGE STACK

- **SQLite 3 (Development & Local Environment):** Zero-configuration, serverless, transactional ACID-compliant single-file database ('donation_tracker.db'). Provides immediate testability and reproducible local execution with zero port bindings.
- **PostgreSQL Dialect Compatibility (Production Readiness):** The entire persistence tier is defined via SQLAlchemy ORM without raw vendor-specific SQL. Transitioning to cloud PostgreSQL requires solely switching the connection URI string ('postgresql://...'), unlocking row-level concurrency, connection pooling ('psycopg2'), and PostGIS spatial extensions.
- **Local Protected File System Vault ('app/static/uploads/')**: Binary images and PDFs are NEVER stored directly as BLOBs in database rows to prevent database bloat. Instead, sanitized files are saved into segmented disk directories ('documents/', 'resources/', 'profile_photos/', 'ngo_gallery/'), while the database stores lightweight string URI paths ('VARCHAR(255)').
- **Cascade Relationship Automation ('cascade=all, delete-orphan')**: Ensures strict referential integrity. When a listing or user is deleted, all dependent chat messages ('Message') and requests ('Request') are purged automatically, eliminating orphaned records.

5. FEATURE-TO-TECHNOLOGY TRACEABILITY MATRIX

Feature Name	Frontend Stack	Backend / Controller Stack	Security / Storage / Algorithmic Layer
User Registration & Auth	HTML5 Forms, Bootstrap 5 Modals, JavaScript	Flask 'auth_routes.py', Flask-Login, Email OTP	Bcrypt (Cost 12), SHA-256 OTP (10 min expiry), 'users' table
DigiLocker e-KYC Verification	DigiLocker Branded Modal UI, SVG Icons	Flask 'auth_routes.py' OAuth 2.0 PKCE Handshake	Cryptographic token verification, verified metadata auto-fill
Hyperlocal Radius Search	Leaflet.js 1.9.4, OpenStreetMap Tiles, Slider UI	Flask 'search_routes.py', 'api_resources()'	Haversine Formula: $d = 2R \cdot \arctan\left(\sqrt{a}, \sqrt{1-a}\right)$ ($R=6,371$ km)
Anti-Hoarding Protection	Bootstrap Dynamic Warning Alert Badges	Flask 'request_routes.py', 'send_request()'	Active Request Counter: $\text{count}(\text{Pending}) < 3$ threshold limit
Anti-Flipping Cooldowns	Dynamic Cooldown Expiry Alert Banners	Flask 'request_routes.py' Temporal Engine	12-Month (365d) Electronics & 6-Month (180d) Appliance Cooldowns
Low-Income (EWS) Gating	Confidential Verified Beneficiary Badge UI	Flask 'profile_routes.py', 'admin_routes.py'	EWS Document Vault; blinded badge protects beneficiary dignity
In-App Handover Messaging	Asynchronous AJAX Chat Window, Auto-Scroll	Flask 'request_routes.py', 'get_messages()'	End-to-End Authorization check; cascade delete-orphan cleanup

Feature Name	Frontend Stack	Backend / Controller Stack	Security / Storage / Algorithmic Layer
Statutory NGO Verification	Document Upload Form (PAN & Auth Letter)	Flask 'admin_routes.py' Approval Engine	Regex: Darpan ('^[A-Z]{2}/d{4}/d{7}\$'), PAN ('^[A-Z]{5}[0-9]{4}[A-Z]{1}\$')
Direct Zero-Cut UPI Support	Dynamic QR Code Renderer, One-Click Intent	Flask 'support_ngos.py', 'UPIDonationClick'	NPCI UPI URI schema: 'upi://pay?pa={upi_id}&pn={name}&cu=INR'
Community ❤️ Like Toggle	Interactive SVG Heart, Asynchronous Counter	Flask 'points_routes.py', '/like-ngo/<id>'	Atomic DB Constraint: 'UniqueConstraint('user_id', 'ngo_id')'
Fraud-Proof Karma Points	Public Leaderboard, Tier Badges (Bronze-Plat)	Flask 'points_routes.py', Points Engine	IP Collision Check (same Wi-Fi $\implies$ 0\$ pts) + 300 pts/7d velocity cap
Mandatory Impact Proof	Camera / File Receipt Upload Interface	Flask 'profile_routes.py', 'upload_impact_proof()'	Receipt Lockout Gate: Blocks new requests if past items unproven

6. INFRASTRUCTURE, VERSION CONTROL & PRODUCTION DEPLOYMENT STACK

- **WSGI Web Server:** PythonAnywhere WSGI Runner / Gunicorn Multi-Worker Server. Spawns $(2 \times \text{CPUs}) + 1$ concurrent worker threads communicating via Unix sockets for parallel request processing.
- **Version Control & Collaboration:** Git version control hosted on GitHub. Modular feature branching, atomic commits, and synchronized production deployment via 'git pull'.
- **Environment & Dependency Isolation:** Python 'virtualenv' managing explicit dependency trees captured in 'requirements.txt' ('Flask', 'SQLAlchemy', 'Flask-Login', 'bcrypt', 'Flask-WTF', 'docx2pdf').